Intelligent Parallel Scaling for High-Performance Workloads

Hugo Wan, Adil Mohiuddin, Sachin Chopra
University of California, Riverside
Email: twan012@ucr.edu, amohi010@ucr.edu, schop021@ucr.edu

***Abstract*—Efficient parallel execution depends not only on adding more threads, but also on selecting an appropriate scheduling strategy, workload partitioning method, and optimization configuration. This paper presents an empirical study of intelligent parallel scaling for dense matrix multiplication using shared-memory parallel programming. The primary implementation is based on C++ and POSIX threads, including a sequential baseline, static scheduling, dynamic scheduling, and an optimized cache-blocked version. An OpenMP implementation is also included as an extension to compare a higher-level shared-memory programming model against the lower-level Pthreads implementation. The experimental evaluation measures average runtime, best runtime, speedup, parallel efficiency, and diminishing returns across multiple matrix sizes and thread counts. Results show that parallel execution significantly improves runtime for larger workloads, but speedup does not scale linearly because of thread overhead, memory behavior, scheduling cost, and cache effects. The study further includes an adaptive policy analysis for identifying near-best configurations without relying only on the maximum available thread count.**

***Index Terms*—parallel computing, Pthreads, OpenMP, matrix multiplication, speedup, efficiency, auto-tuning, shared-memory systems**

I. Introduction

Shared-memory parallel programming is widely used to accelerate computationally intensive workloads on multicore processors. However, practical performance scaling is not guaranteed simply by increasing the number of threads. Parallel programs must balance computation, synchronization overhead, memory locality, scheduling strategy, and resource usage. If too few threads are used, the program may underutilize available cores. If too many threads are used, the program may experience overhead, contention, cache pressure, or diminishing returns.

Dense matrix multiplication is a useful workload for studying these effects because it is computationally intensive, regular in structure, and sensitive to memory-access behavior. It also provides a clear baseline for evaluating runtime, speedup, and efficiency. Although the mathematical operation is straightforward, implementation choices strongly affect performance on real hardware.

This work studies intelligent parallel scaling for dense matrix multiplication. The primary implementation uses C++ with POSIX threads, referred to as Pthreads in this paper. The Pthreads implementation includes four execution modes: a sequential baseline, static row partitioning, dynamic thread selection, and an optimized cache-blocked implementation. The project also includes an OpenMP extension that provides a higher-level implementation of similar static and dynamic scheduling ideas.

The main contributions of this work are as follows:

- A complete Pthreads-based matrix multiplication pipeline with sequential, static, dynamic, and optimized execution modes.
- A benchmarking workflow for collecting average runtime, best runtime, speedup, and efficiency across multiple matrix sizes and thread counts.
- An adaptive policy analysis for selecting near-best configurations under a runtime tolerance.
- A saturation analysis for studying diminishing returns in thread count and block-size selection.
- An OpenMP extension for comparing the lower-level Pthreads implementation with a directive-based shared-memory programming model.

II. Background

A. Shared-Memory Parallelism

In shared-memory systems, multiple threads execute concurrently and access a common address space. This model simplifies data sharing but introduces performance and correctness concerns such as synchronization, load imbalance, cache behavior, and scheduling overhead. Pthreads provides a low-level interface for explicitly creating, joining, and managing threads. This gives programmers direct control over thread behavior, but it also requires manual workload partitioning and careful implementation.

OpenMP provides a higher-level shared-memory programming model using compiler directives. Instead of manually creating threads, programmers annotate loops or regions of code and allow the OpenMP runtime to manage parallel execution. This can reduce implementation complexity, but performance still depends on scheduling choices, thread count, and workload structure.

B. Performance Metrics

The main performance metrics used in this work are runtime, speedup, and parallel efficiency. Runtime measures wall-clock execution time. Speedup compares sequential runtime against parallel runtime:

$$S_p = \frac{T_1}{T_p}, \tag{1}$$

where T_1 is the sequential runtime and T_p is the runtime using p threads. Parallel efficiency measures how effectively the threads are used:

$$E_p = \frac{S_p}{p}. \tag{2}$$

In ideal scaling, speedup would increase linearly with the number of threads, and efficiency would remain close to 1. In practice, overhead and memory effects usually reduce efficiency as thread count increases.

C. Diminishing Returns

A key issue in parallel scaling is diminishing returns. Adding more threads may continue to reduce runtime, but the improvement can become small relative to the additional resource cost. For example, moving from two to four threads may significantly reduce runtime, while moving from eight to sixteen threads may provide only a small gain. A practical parallel policy should therefore consider both the fastest configuration and the lowest-cost configuration that remains close to the best runtime.

III. Implementation

A. Primary Pthreads Implementation

The primary implementation is written in C++ using Pthreads. The workload is dense square matrix multiplication. Given matrices A, B, and C, the program computes:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}. \tag{3}$$

The implementation includes the following execution modes.

1) Sequential Baseline: The sequential version performs matrix multiplication using a single thread. It serves as both the correctness reference and the baseline for speedup calculation.

2) Static Pthreads Scheduling: The static version creates a fixed number of Pthreads and divides matrix rows among them before computation begins. Each thread receives a fixed range of rows. This approach has low scheduling overhead and works well when the workload is evenly distributed.

3) Dynamic Pthreads Scheduling: The dynamic version selects a thread count automatically based on matrix size. This mode represents the idea that a single fixed thread count may not be appropriate for all workload sizes. Smaller matrices may not benefit from many threads because thread overhead can dominate. Larger matrices can usually benefit from more parallelism.

4) Optimized Cache-Blocked Pthreads: The optimized version adds cache blocking to improve memory locality. Instead of computing the entire multiplication using only the basic triple-loop structure, the computation is organized into blocks. This can reduce cache misses by reusing data within smaller working sets. The optimized implementation also supports configurable block size for additional tuning.

B. OpenMP Extension

In addition to the primary Pthreads implementation, the project includes an OpenMP extension. The OpenMP implementation provides sequential, static, and dynamic execution modes using OpenMP directives and runtime support. The extension is useful for comparing manual thread management against a higher-level parallel programming model.

The OpenMP dynamic mode uses a tuning phase to test candidate thread counts on a sample workload. It then selects a thread count according to either a performance-oriented goal or an efficiency-oriented goal. This design allows the OpenMP extension to study the same general problem as the Pthreads implementation: selecting a thread count that balances performance and resource usage.

IV. Experimental Methodology

A. Workload Sizes and Thread Counts

The Pthreads experiments evaluate matrix sizes $N = 256$, 512, 1024, 1536, and 2048. For static and optimized modes, the tested thread counts include 1, 2, 4, and 8 threads. The dynamic mode selects a thread count automatically. Each configuration is executed multiple times, and both average runtime and best runtime are recorded.

B. Measured Quantities

The collected results include:

- average runtime in seconds,
- best runtime in seconds,
- speedup computed from average runtime,
- efficiency computed from average runtime,
- selected configuration from the adaptive policy,
- saturation behavior for thread count and block size.

Average runtime is used for the primary speedup and efficiency analysis because it reflects typical observed performance across repeated trials. Best runtime is also recorded to show the strongest observed execution time under lower noise.

C. Adaptive Policy

The adaptive policy uses measured results to select a practical configuration. For each matrix size, the policy first identifies the fastest measured static or optimized configuration. It then finds configurations within a specified tolerance of the fastest runtime. In this study, the tolerance is 10%. Among the near-best configurations, the policy selects the one that uses fewer resources when possible.

This policy is designed to avoid blindly selecting the maximum thread count when a lower-cost configuration provides nearly the same runtime. In the collected results, the fastest configuration was often still the 8-thread static configuration, but the policy framework remains useful for analyzing performance-resource tradeoffs.

V. Results

A. Pthreads Runtime and Speedup

Table I summarizes representative Pthreads results across matrix sizes. The table compares the sequential baseline, best static Pthreads result, dynamic Pthreads result, and best optimized result. The speedup values are computed from average runtime.

The results show that parallel execution provides clear runtime improvement over the sequential baseline. For smaller matrices, speedup is limited because the workload is not large enough to fully amortize thread creation, synchronization, and

TABLE I: Representative Pthreads Results Across Matrix Sizes

Matrix Size N	Mode	Threads	Avg. Runtime (s)	Best Runtime (s)	Speedup
256	Sequential	1	0.016	0.015	1.000
	Static	8	0.004	0.004	3.917
	Dynamic	4	0.005	0.005	3.133
	Optimized	8	0.005	0.004	3.133
512	Sequential	1	0.118	0.118	1.000
	Static	8	0.024	0.023	5.000
	Dynamic	8	0.024	0.023	5.000
	Optimized	8	0.027	0.026	4.438
1024	Sequential	1	0.990	0.965	1.000
	Static	8	0.177	0.169	5.604
	Dynamic	8	0.174	0.167	5.701
	Optimized	8	0.183	0.181	5.420
1536	Sequential	1	5.824	5.417	1.000
	Static	8	0.663	0.648	8.780
	Dynamic	8	0.652	0.643	8.928
	Optimized	8	0.691	0.681	8.425
2048	Sequential	1	14.125	13.910	1.000
	Static	8	1.671	1.600	8.453
	Dynamic	8	1.614	1.602	8.754
	Optimized	8	1.715	1.670	8.238

TABLE II: Pthreads Efficiency for Larger Matrix Sizes

N	Mode	Threads	Efficiency
1024	Static	8	0.700
1024	Dynamic	8	0.713
1536	Static	8	1.098
1536	Dynamic	8	1.116
2048	Static	8	1.057
2048	Dynamic	8	1.094

TABLE III: Dynamic Pthreads Compared with Best Static and Optimized Configurations

N	Dynamic Avg. (s)	Best Static Avg. (s)	Best Optimized Avg. (s)
256	0.005	0.004	0.005
512	0.024	0.024	0.027
1024	0.174	0.177	0.183
1536	0.652	0.663	0.691
2048	1.614	1.671	1.715

scheduling overhead. For larger matrices, speedup improves substantially because the computation dominates overhead. At $N = 2048$, dynamic Pthreads achieves an average runtime of 1.614 seconds compared with 14.125 seconds for the sequential baseline, corresponding to an average-runtime speedup of 8.754.

B. Efficiency Behavior

Table II summarizes the best static and dynamic Pthreads efficiency values for larger matrix sizes. Efficiency above 1 can appear in empirical measurements when the parallel implementation benefits from cache effects, measurement variation, or when the single-thread baseline is not identical in memory behavior to the parallel configuration. Therefore, these values should be interpreted as empirical efficiency relative to the measured baseline rather than as ideal theoretical efficiency.

The efficiency results suggest that larger workloads make better use of parallel resources. For $N = 256$ and $N = 512$, overhead has a stronger impact, while $N = 1536$ and $N = 2048$ show stronger scaling. This is consistent with the expectation that larger computational workloads provide more work per thread and reduce the relative cost of scheduling overhead.

C. Static, Dynamic, and Optimized Comparison

Table III compares dynamic Pthreads against the best measured static and optimized configurations. The dynamic mode is competitive with the best static mode for medium and large matrices. For $N = 1024$, 1536, and 2048, dynamic Pthreads slightly outperforms the best static result in average runtime. For smaller matrices, the difference is small and can be affected by measurement noise.

The optimized cache-blocked version did not always outperform the basic static implementation in the summarized results. This does not mean that cache blocking is ineffective in general. Instead, it indicates that the chosen block sizes, matrix sizes, compiler behavior, and hardware environment all affect whether blocking produces a measurable benefit. The saturation experiments further examine this issue by testing different block sizes.

D. Adaptive Policy Results

Table IV shows the adaptive policy output using a 10% runtime tolerance. Here, Ref. denotes the reference configuration, Sel. denotes the adaptive policy selection, and p denotes the number of threads.

The policy selected the static 8-thread configuration for all tested matrix sizes because it remained within the near-best runtime threshold. Although the dynamic mode was slightly

TABLE IV: Adaptive Policy Selection

N	Reference	Ref. p	Selected	Sel. p
256	Static	8	Static	8
512	Static	8	Static	8
1024	Static	8	Static	8
1536	Static	8	Static	8
2048	Static	8	Static	8

TABLE V: Representative Saturation and Auto-Tuning Results

Search Type	N	Configuration	Avg. Runtime (s)
Linear thread	256	8 threads, block 32	0.008333
Linear thread	512	16 threads, block 32	0.034667
Linear thread	1024	16 threads, block 32	0.170333
Hill-climb block	256	block 32, 8 threads	0.007000
Hill-climb block	512	block 32, 16 threads	0.030667
Hill-climb block	1024	block 128, 16 threads	0.153333

faster for some larger matrix sizes, the static configuration remained a simple and consistently strong reference configuration.

Although the selected thread count did not decrease in this dataset, the adaptive policy remains valuable because it provides a general framework for identifying resource-efficient configurations. On different hardware, larger thread-count ranges, or more irregular workloads, the same policy could select a lower thread count when additional threads provide only marginal runtime improvement.

E. Saturation and Auto-Tuning Results

The saturation workflow studies how runtime changes as thread count or block size increases. Table V shows representative results from the optimized thread-scaling and block-size tuning experiments.

These results show that tuning can improve performance beyond a fixed default configuration. For example, in the block-size search for $N = 1024$, the tested block size of 128 achieved the lowest average runtime among the listed block sizes. This supports the idea that practical performance tuning should consider both thread count and memory-locality parameters.

VI. Discussion

A. Why Larger Matrices Scale Better

The results show stronger speedup for larger matrices. This occurs because matrix multiplication has cubic computational complexity with respect to matrix dimension. As N increases, the amount of arithmetic work grows rapidly. The fixed overheads of thread creation, scheduling, joining, and measurement become less significant relative to the total computation. Therefore, larger workloads better amortize parallel overhead.

B. Static Versus Dynamic Scheduling

Static scheduling performs well for dense matrix multiplication because the workload is regular. Each row requires roughly the same amount of computation, so assigning fixed row ranges to threads usually gives balanced work distribution. Dynamic scheduling is more useful when workload cost varies across tasks. In this project, dynamic Pthreads remained competitive because it selected appropriate thread counts based on matrix size, but the scheduling benefit was limited by the regular structure of dense matrix multiplication.

C. Effect of Cache Blocking

Cache blocking is intended to improve locality by reusing matrix data within smaller blocks. However, the measured optimized results show that blocking is not automatically faster in every configuration. A blocked implementation may introduce additional loop overhead or may use a block size that does not match the cache hierarchy well. The saturation results show that block-size tuning matters, especially for larger matrices. This suggests that optimization should be treated as a parameterized search problem rather than a single fixed implementation.

D. Pthreads and OpenMP Comparison

The Pthreads implementation provides fine-grained control over thread creation, work partitioning, and execution behavior. This makes it useful for understanding the mechanics of parallel scaling. However, it also requires more manual implementation effort.

The OpenMP extension provides a more concise implementation using compiler directives and runtime support. It makes it easier to express static and dynamic parallelism, but the same performance questions remain: how many threads should be used, what scheduling policy should be selected, and when does additional parallelism stop helping? In this sense, OpenMP reduces programming complexity but does not remove the need for empirical tuning.

E. Practical Implications

The main practical implication is that high-performance parallel programs should not assume that maximum thread count is always best. A good implementation should measure performance, evaluate efficiency, and identify diminishing returns. The adaptive policy and saturation analysis provide a step toward this goal by treating configuration selection as a data-driven process.

VII. Limitations

This study has several limitations. First, the workload is primarily dense matrix multiplication, which is regular and compute-heavy. More irregular workloads may show stronger benefits from dynamic scheduling. Second, the experiments were conducted on a limited hardware environment, so exact runtime values may differ across machines. Third, the optimized implementation uses a finite set of tested block sizes and thread counts. A broader search space could identify stronger configurations. Fourth, the OpenMP implementation is included as an extension and comparison point, while the most detailed quantitative tables in this paper focus on the Pthreads pipeline.

VIII. Conclusion

This paper presented an empirical study of intelligent parallel scaling for high-performance workloads using dense matrix multiplication. The primary implementation used C++ and Pthreads with sequential, static, dynamic, and optimized execution modes. An OpenMP extension was also included to compare a higher-level parallel programming model with the lower-level Pthreads implementation.

The results show that parallel execution substantially reduces runtime for larger matrices. For $N = 2048$, dynamic Pthreads reduced average runtime from 14.125 seconds to 1.614 seconds, achieving a speedup of 8.754. Static scheduling performed strongly because dense matrix multiplication has regular work distribution, while dynamic selection remained competitive for larger matrix sizes. Cache-blocking performance depended on block-size choice, showing the importance of tuning memory-locality parameters. Finally, the adaptive policy and saturation analysis demonstrated how performance measurement can guide configuration selection beyond simply using the maximum thread count.

Overall, the study shows that effective parallel scaling requires both implementation-level optimization and empirical configuration analysis. Thread count, scheduling strategy, cache behavior, and workload size must be considered together to achieve practical high-performance execution.

Contribution Summary

Table VI summarizes the team contribution distribution. The percentages reflect the visible implementation, experimental, documentation, presentation, and final packaging work completed for the project.

TABLE VI: Author Contribution Summary

Author	Main Contributions	Percentage
Hugo Wan	Primary project direction, proposal organization, main Pthreads implementation, sequential/static/dynamic/optimized pipeline, benchmarking scripts, result collection, plotting utilities, adaptive policy analysis, saturation analysis, documentation, report preparation, presentation preparation, and final submission packaging.	70%
Adil Mohiuddin	OpenMP extension direction and support, implementation/testing support, multi-run averaging and error-bar plotting support, repository cleanup support, presentation support, and project discussion.	20%
Sachin Chopra	Additional workload support, implementation/testing support, presentation support, and project discussion.	10%

References

[1] IEEE and The Open Group, "POSIX Threads," The Open Group Base Specifications, Issue 7, 2018.

[2] OpenMP Architecture Review Board, "OpenMP Application Programming Interface Version 5.2," 2021.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 1967, pp. 483–485.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.